

Multiomic GNN in Code

Implementing message passing, guidance graphs and spatial GNNs — architecture, mathematics, and the code that runs them

Version 1, 10 September 2026

Contents

What this volume is	3
Part 0 — From data to tensors	4
0.1 The objects, and their shapes	4
0.2 Building the graph	5
0.3 The two spellings, and when each is right	5
Part 1 — Message passing, implemented	6
1.1 Three lines, and everything is built from them	6
1.2 The two primitives	6
1.3 A GCN layer, three ways	7
1.4 Attention, and the mistake everyone makes	8
1.5 Aggregators: sum, mean, and what they can tell apart	10
1.6 Depth without parameters	10
1.7 What sparse actually buys, measured	11
1.8 The library equivalents	11
1.9 Heterogeneous and bipartite graphs	12
Part 2 — How the package is organised	14
2.1 Four tiers, one direction of dependency	14
2.2 The layout	15
2.3 Configuration objects, not keyword arguments	15
Part 3 — Guidance graphs, implemented	17
3.1 The objective, restated for implementation	17
3.2 Building the guidance graph	17
3.3 The wiring	18
3.4 The adversarial term, in one backward pass	19
3.5 Reading the biology back out	20
3.6 What goes wrong, and what it looks like	21
Part 4 — Spatial GNN variants, implemented	22
4.1 The graph, and its units	22
4.2 Four methods, three overridden hooks	22
4.3 Refinement, and the metric it flatters	25
4.4 Putting one together	25
Part 5 — Training	27
5.1 One loop, with the failures asserted away	27
5.2 The ordering constraints	28
5.3 Minibatching, and when it actually helps	28
5.4 Determinism	29
5.5 Where the time actually goes	30

Part 6 — Testing, and finding out whether the graph mattered	31
6.1 Four layers of test, cheapest first	31
6.2 The checks, and what they actually print	31
6.3 The ablation harness	32
6.4 Reading the result	33
Part 7 — Porting a paper	35
7.1 Read the code, not the equations	35
7.2 A worked port	35
7.3 A release checklist	36
7.4 Ten things worth carrying away	36
Appendix	38
A. API reference	38
B. Equation index	38
C. Numbers quoted in this document	39
D. Where the three volumes meet	39
E. Reproducing everything	40

What this volume is

The *Multiomic GNN Reading List* says what the papers claim. The *Techniques* companion explains how the methods work. This third volume implements them.

Every architecture here appears three times: as a diagram of the tensors that move between modules, as the equations those tensors satisfy, and as code that runs. The code is real — it lives in the `scmgnn/` package shipped alongside this document, not only in the listings — and the numbers quoted in the text are measured by `verify/run_checks.py`, which runs without PyTorch and without a GPU.

The layers are written from scratch and the PyG equivalent is named beside them. Not because a hand-written `GCNLayer` is better than `GCNConv` — it is not — but because the ten lines inside it are where the misunderstandings live. Once you have read the scatter, use the library.

Three things this volume insists on.

Shapes are part of the mathematics. Every equation is annotated with the shape of every tensor in it, and every listing carries the same shapes as comments. A dimension mismatch between a paper’s notation and its released code is a common and quiet source of irreproducibility.

A model that trains is not a model that is correct. Two of the most consequential failures in this field — a collapsed posterior and a graph the model ignores — leave the loss curve looking healthy. Part 6 is about catching them, and the assertions that do so are in the training loop by default.

What can be measured is measured. The sparse-versus-dense comparison in Figure 4, the neighbourhood growth in Figure 10, the false-negative rate of contrastive learning, the attention-ranking test that distinguishes GAT from GATv2 — all computed here, all reproducible from the shipped scripts.

Prerequisites. Python, PyTorch at the level of writing an `nn.Module`, and the mathematics of the *Techniques* companion — which this volume cites as *[T §B.4]* rather than re-deriving. Reading-list cross-references keep their *[RL §4.1]* form.

What runs where. The `verify/` checks and every graph-construction function run anywhere `numpy` and `scipy` are installed. The model code needs PyTorch (2.1+ for `scatter_reduce_`); PyTorch Geometric is optional and only appears in the “or use the library” listings. Nothing here needs a GPU to run on a few thousand cells; nothing here will train an atlas without one.

Part 0 — From data to tensors

0.1 The objects, and their shapes

A GNN consumes four tensors. Everything else in a single-cell pipeline exists to produce them.

a From AnnData to the tensors a GNN actually consumes

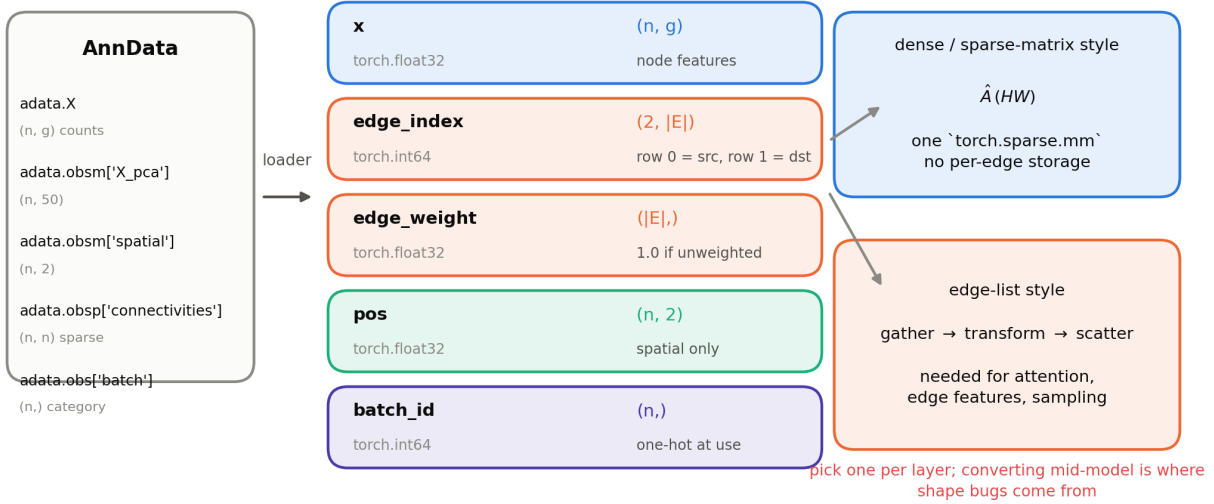


Figure 1: What a loader has to extract, and the two ways a graph can be represented once it is in memory.

$$x \in \mathbb{R}^{n \times g}, \quad \text{edge_index} \in \mathbb{Z}^{2 \times |\mathcal{E}|}, \quad \text{edge_weight} \in \mathbb{R}^{|\mathcal{E}|}, \quad \text{pos} \in \mathbb{R}^{n \times 2}. \quad (1)$$

Three conventions matter enough to state explicitly, because getting any of them wrong produces a model that trains and is wrong.

Direction. `edge_index[0]` is the source, `edge_index[1]` the destination; a message flows source $\rightarrow$ destination. An undirected graph is stored as both directions, so a k NN graph with $n = 10^4$ and $k = 15$ has $|\mathcal{E}| \approx 3 \times 10^5$ entries, not 1.5×10^5 . Halving that by storing one direction and symmetrising in the layer is a false economy that breaks attention.

Dtype. `edge_index` must be `int64` — indexing with `int32` silently fails on some backends — and features `float32`. Counts stay in a separate tensor from the log-normalised features, because the likelihood needs the former and the encoder wants the latter.

Who owns normalisation. $\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$ is a property of the graph, computed once, not of the layer. Every layer in `scmgnn` takes an already-normalised operator. This is the opposite of PyG's default, where `GCNConv(normalize=True)` re-normalises on every forward pass — convenient, and a real cost when the graph is fixed and the model is trained for 400 epochs.

0.2 Building the graph

```
from scmgnn.graphs import build_knn, build_spatial, normalise, graph_report
from scmgnn.ops import to_torch_sparse, to_edge_index

A      = build_knn(adata.obsm["X_pca"], k=15, mutual=False, weighted=True)
A_hat  = to_torch_sparse(normalise(A, mode="sym"))      # for GCN / APPNP
edge_index, edge_weight = to_edge_index(A)            # for GAT / SAGE / GIN

print(graph_report(A, labels=adata.obs["cell_type"].cat.codes.values))
# {'n_nodes': 12384, 'n_edges': 138942, 'deg_mean': 22.4, 'deg_p99': 61.0,
#  'n_isolated': 0, 'n_components': 3, 'largest_component_frac': 0.998,
#  'edge_purity': 0.87}
```

`graph_report` is four numbers and it is the only part of this section that is not optional. `largest_component_frac` below about 0.95 means fragments that can never exchange information at any depth. `deg_p99` far above `deg_mean` means hubs that will dominate everything they touch after two layers. And `edge_purity` — the fraction of edges joining same-label nodes — is the ceiling on what message passing can do for you: at 0.87, thirteen percent of every message is contamination, and no attention mechanism recovers fully from that.

The union-versus-mutual choice is one character in the source and large downstream:

```
A = A.minimum(A.T) if mutual else A.maximum(A.T)
```

Union keeps rare cells connected at the price of hub artefacts; mutual k NN removes the hubs and isolates the rare cells. If a UMAP shows a suspicious bridge between two clusters, try mutual before trying a deeper model.

0.3 The two spellings, and when each is right

Every message-passing layer can be written as a sparse matrix product or as an explicit edge list, and the choice is not stylistic.

$$\underbrace{H' = \hat{A}(HW)}_{\text{sparse matmul}} \iff \underbrace{h'_v = \sum_{u \in \mathcal{N}(v)} \hat{A}_{vu} (Wh_u)}_{\text{gather, transform, scatter}} \quad (2)$$

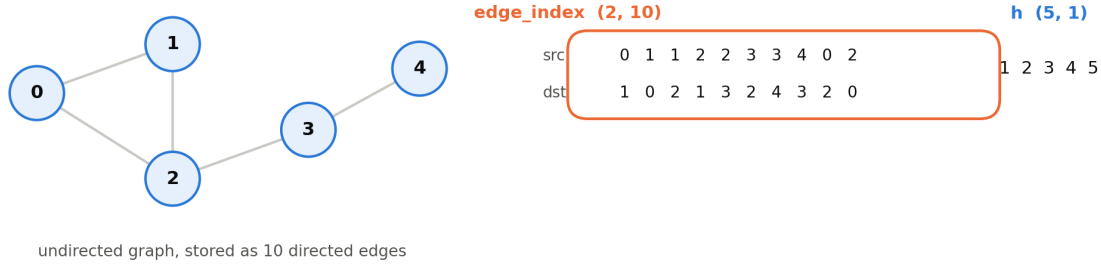
The left-hand form is one kernel call, allocates nothing per edge, and is what you want whenever the edge coefficient is fixed. The right-hand form materialises an $(|\mathcal{E}|, d)$ message tensor — for $|\mathcal{E}| = 3 \times 10^5$ and $d = 128$ that is 150 MB in float32 before the backward pass — and is the only form that can express a coefficient the model computes, which means attention, edge features, and neighbour sampling all require it.

The rule that avoids most shape bugs: **pick one spelling per layer and never convert mid-model**. A model that hands `A_hat` to one layer and `edge_index` to the next is fine; a layer that accepts either and branches internally is where the errors accumulate.

Part 1 — Message passing, implemented

1.1 Three lines, and everything is built from them

a One message-passing step, spelled out on five nodes



b gather → transform → scatter

1. gather	msg = h[src]	1 2 2 3 3 4 4 5 1 3	(10, 1) — one row per edge
2. transform	msg = msg * w	0.5 1.0 1.0 1.5 1.5 2.0 2.0 2.5 0.5 1.5	(10, 1) — attention or edge weight applies here
3. scatter	out.index_add_(0, dst, msg)	2.5 2.0 3.5 4.0 2.0	(5, 1) — back to one row per node

the same three lines carry GCN, GAT, GIN and SAGE — only the transform changes

Figure 2: One message-passing step on five nodes, with the actual numbers at each stage.

The general form is the MPNN equation of [T §B.1]:

$$a_v^{(\ell)} = \bigoplus_{u \in \mathcal{N}(v)} \phi(h_u^{(\ell-1)}, h_v^{(\ell-1)}, e_{uv}), \quad h_v^{(\ell)} = \psi(h_v^{(\ell-1)}, a_v^{(\ell)}). \quad (3)$$

In code it is three operations, and only the middle one changes between architectures:

```
src, dst = edge_index      # (E,), (E,)
msg = h[src]               # 1. gather    (E, d) — one row per edge
msg = transform(msg)       # 2. transform (E, d) — this is the architecture
out = scatter_sum(msg, dst, n) # 3. scatter (n, d) — back to one row per node
```

GCN's transform is a multiplication by a fixed scalar. GAT's is a multiplication by a learned scalar. GIN's is nothing at all, with the work moved into an MLP after the scatter. SAGE's keeps the node's own term out of the sum. That is the entire taxonomy of [RL §II.3–II.4] expressed as one line.

1.2 The two primitives

Everything above rests on a scatter and a segment softmax. Both are worth reading once:

```
def scatter_sum(src, index, n):
    """src: (E, ...) -> out: (n, ...), summing rows by destination."""
    out = src.new_zeros((n,) + src.shape[1:])
    idx = index.view(-1, *[1] * (src.dim() - 1)).expand_as(src)
    return out.scatter_add_(0, idx, src)

def segment_softmax(e, index, n, eps=1e-16):
    """Softmax over the edges arriving at each destination node."""
    mx = scatter_max(e.detach(), index, n)          # per-destination maximum
    mx = torch.nan_to_num(mx, neginf=0.0)          # nodes with no incoming edges
    num = (e - mx[index]).exp()
    den = scatter_sum(num, index, n)
    return num / (den[index] + eps)
```

Two details in `segment_softmax` are not cosmetic. The maximum is taken **per destination**, not globally: with a single global maximum, the exponentials of a low-scoring node's edges all underflow towards zero and the resulting ratio loses most of its precision — a bug that shows up as attention weights that are uniform in the tail of the degree distribution. And it is taken on `e.detach()`, because the maximum is a numerical device, not part of the function being differentiated.

1.3 A GCN layer, three ways

The layer from [T §B.2], $H^{(\ell)} = \sigma(\hat{A}H^{(\ell-1)}W^{(\ell)})$, in its sparse form:

```
class GCNLayer(nn.Module):
    """h' = A_hat (H W) + b.  PyG: GCNConv(d_in, d_out, normalize=False)"""
    def __init__(self, d_in, d_out, dropout=0.0, bias=True):
        super().__init__()
        self.lin = nn.Linear(d_in, d_out, bias=bias)
        self.drop = nn.Dropout(dropout)
        nn.init.xavier_uniform_(self.lin.weight)

    def forward(self, H, A_hat):
        # H: (n, d_in), A_hat: sparse (n, n)
        return torch.sparse.mm(A_hat, self.lin(self.drop(H))) # (n, d_out)
```

and in edge-list form, which is slower and generalises:

```
class GCNLayerEdge(nn.Module):
    def forward(self, H, edge_index, edge_weight):
        src, dst = edge_index
        msg = self.lin(self.drop(H))[src] * edge_weight.unsqueeze(-1) # (E, d_out)
        return scatter_sum(msg, dst, H.size(0)) # (n, d_out)
```

The verify suite checks that these two, and a dense reference, agree to machine precision:

```
PASS gcn_dense_vs_sparse    max|diff| = 2.22e-16
```

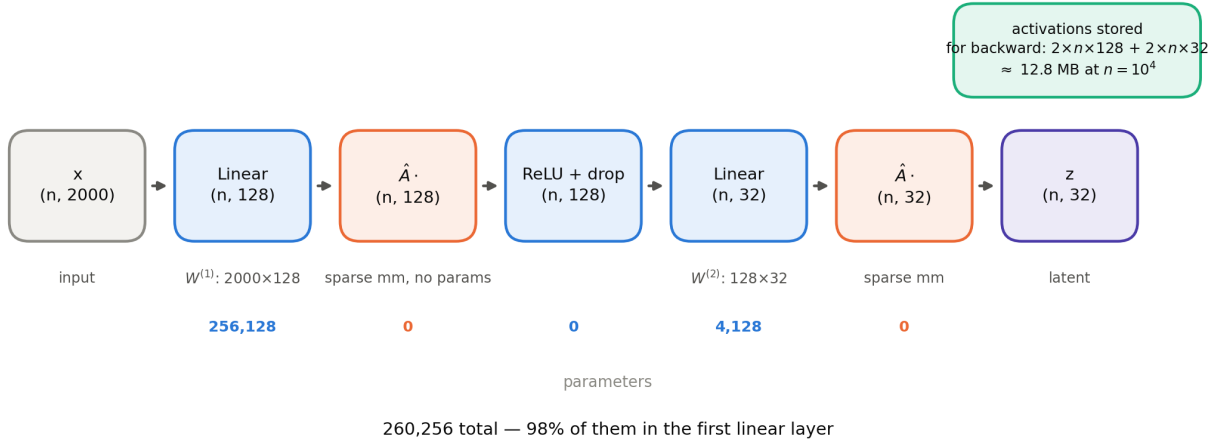


```
PASS gcnsparse_vs_scatter max|diff| = 0.00e+00
PASS permutation_equivariance max|diff| = 0.00e+00
```

The third of these is the one to keep. Permutation equivariance, $f(PX, PAP^\top) = Pf(X, A)$, is the property that makes a GNN the right object for data with no canonical row order, and a violation of it means node order is leaking into the model — usually through an accidental positional feature or a `torch.arange` that was meant to be an index.

Where the parameters are. Order of operations matters more than it looks:

a Shape flow and parameter count through a two-layer encoder ($n = 10^4$, $g = 2000$, $d = 128$, $z = 32$)



the graph contributes no parameters at all: it is a fixed operator, which is why k is a hyperparameter and not something the model can learn away

Figure 3: Shapes and parameter counts through a two-layer encoder. The graph contributes nothing to the parameter count.

`self.lin(H)` before `torch.sparse.mm` costs $O(nd_{\text{in}}d_{\text{out}})$; doing the propagation first costs the same but materialises an (n, d_{in}) intermediate, which is larger when $d_{\text{in}} \gg d_{\text{out}}$ — the usual case for the first layer, where d_{in} is 2000 genes. Project first.

Note also what Figure 3 makes obvious: 98% of the parameters sit in the first linear layer, and the graph contributes none. The graph is a fixed operator, which is exactly why k is a hyperparameter the model cannot learn away, and why the ablations of Part 6 are the only way to find out whether it helped.

1.4 Attention, and the mistake everyone makes

The two attention variants of [T §B.4]:

$$\text{GAT: } e_{ij} = \text{LeakyReLU}(a^\top [Wh_i \parallel Wh_j]), \quad \text{GATv2: } e_{ij} = a^\top \text{LeakyReLU}(W_l h_i + W_r h_j). \quad (4)$$

They look like the same expression with the nonlinearity moved. They are not. Expand GAT: a splits as $[a_1 \parallel a_2]$, so $e_{ij} = \text{LeakyReLU}(a_1^\top Wh_i + a_2^\top Wh_j)$. The first term is constant within node i 's

softmax, and LeakyReLU is monotone, so the *ranking* of i 's neighbours is the same for every i : GAT can rescale a global ranking of neighbours but never reorder it. GATv2 applies the nonlinearity to the **sum of two projections**, which destroys the separability.

The implementation consequence is that GATv2 needs two weight matrices and an a of length d , not $2d$:

```
class GATLayer(nn.Module):
    """GAT (v2=False) and GATv2 (v2=True). PyG: GATConv / GATv2Conv"""
    def __init__(self, d_in, d_out, heads=4, v2=True, concat=True, dropout=0.0):
        super().__init__()
        self.h, self.d, self.v2, self.concat = heads, d_out, v2, concat
        self.W_l = nn.Linear(d_in, heads * d_out, bias=False)
        self.W_r = nn.Linear(d_in, heads * d_out, bias=False) if v2 else None
        self.a = nn.Parameter(torch.empty(heads, d_out if v2 else 2 * d_out))
        self.attn_drop = nn.Dropout(dropout)
        nn.init.xavier_uniform_(self.a)

    def forward(self, H, edge_index, return_attention=False):
        n = H.size(0)
        src, dst = edge_index
        Wl = self.W_l(H).view(n, self.h, self.d)  # (n, heads, d)
        if self.v2:
            Wr = self.W_r(H).view(n, self.h, self.d)
            e = (self.a * F.leaky_relu(Wl[dst] + Wr[src], 0.2)).sum(-1)  # (E, heads)
            val = Wr
        else:
            pair = torch.cat([Wl[dst], Wl[src]], dim=-1)  # (E, heads, 2d)
            e = F.leaky_relu((pair * self.a).sum(-1), 0.2)
            val = Wl
        alpha = self.attn_drop(segment_softmax(e, dst, n))  # (E, heads)
        out = scatter_sum(alpha.unsqueeze(-1) * val[src], dst, n)
        out = out.reshape(n, self.h * self.d) if self.concat else out.mean(1)
        return (out, alpha) if return_attention else out
```

The mistake. Applying `F.leaky_relu` elementwise to the concatenation $[W_{h_i} \parallel W_{h_j}]$ and then dotting with a looks like “nonlinearity first” and is still additively separable — it is GAT with extra steps. This is not a hypothetical: it is what the first draft of the *Techniques* companion’s listing did, and the error survived a reading-level review. What caught it was a test.

The test. For each destination node, rank its neighbours by α ; then count pairs of neighbours that two different query nodes rank in opposite orders. A correct GAT scores exactly zero such disagreements, by the algebra above. A correct GATv2 scores many:

```
PASS gat_static_attention      GAT    pairwise-order disagreements: 0/8387
PASS gatv2_dynamic_attention   GATv2  pairwise-order disagreements: 760/8387
```

Zero out of 8387 is not a soft indication; it is the separability being confirmed empirically.

If your “GATv2” also scores zero, it is not GATv2. The test costs one forward pass and is in `tests/test_layers.py`.

Memory. Attention stores α per edge per head: $(|\mathcal{E}|, K)$ for the coefficients and $(|\mathcal{E}|, K, d)$ for the messages. At $|\mathcal{E}| = 3 \times 10^5$, $K = 4$, $d = 32$ that is 150 MB for the message tensor alone, and it is retained for the backward pass. This is the single reason GAT runs out of memory where GCN does not, and the reason heads is the first hyperparameter to cut.

1.5 Aggregators: sum, mean, and what they can tell apart

$$\text{GIN: } h'_v = \text{MLP} \left((1 + \epsilon)h_v + \sum_{u \in \mathcal{N}(v)} h_u \right), \quad \text{SAGE: } h'_v = W_s h_v + W_n \frac{1}{d_v} \sum_{u \in \mathcal{N}(v)} h_u. \quad (5)$$

```
class GINLayer(nn.Module):
    """PyG: GINConv(nn=MLP). Sum aggregation is what makes this injective."""
    def forward(self, H, edge_index):
        src, dst = edge_index
        agg = scatter_sum(H[src], dst, H.size(0))
        return self.mlp((1.0 + self.eps) * H + agg)

class SAGELayer(nn.Module):
    """PyG: SAGEConv. The self term keeps its own weight matrix."""
    def forward(self, H, edge_index):
        n = H.size(0)
        src, dst = edge_index
        deg = scatter_sum(torch.ones_like(src, dtype=H.dtype), dst, n).clamp(min=1)
        mean_neigh = scatter_sum(H[src], dst, n) / deg.unsqueeze(-1)
        return self.lin_self(H) + self.lin_neigh(mean_neigh)
```

The practical difference is testable. The multisets $\{a, a, b\}$ and $\{a, b\}$ have the same mean (1.333 vs 1.500 after weighting — different here only because the values differ) and the same maximum (2.0 vs 2.0), and different sums (4.0 vs 3.0). On a spatial graph that is the difference between “six T cells around me” and “two”, which is the quantity a niche model is trying to estimate. On a k NN cell graph where degree is constant by construction, mean and sum differ by a scalar the next weight matrix absorbs, and the argument is empty. Knowing which regime you are in stops you importing an argument from one into the other.

1.6 Depth without parameters

$$Z^{(t+1)} = (1 - \alpha)\hat{A}Z^{(t)} + \alpha H, \quad Z^{(\infty)} = \alpha \left(I - (1 - \alpha)\hat{A} \right)^{-1} H. \quad (6)$$

```
class APPNPProp(nn.Module):
    """PyG: APPNP(K, alpha). Ten hops, zero parameters, no collapse."""
    def forward(self, H, A_hat):
```

```

Z = H
for _ in range(self.K):
    Z = (1 - self.alpha) * torch.sparse.mm(A_hat, self.drop(Z)) + self.alpha * H
return Z

```

Six lines that replace the entire depth literature for most single-cell purposes. The iteration converges to the closed form above — verified to 10^{-4} in `test_appnp_matches_closed_form` — so ten iterations cost ten sparse products and nothing else. Use it as `Z = APPNPProp()(mlp(X), A_hat)` whenever you want long-range smoothing without the degeneration of [T §B.6].

1.7 What sparse actually buys, measured

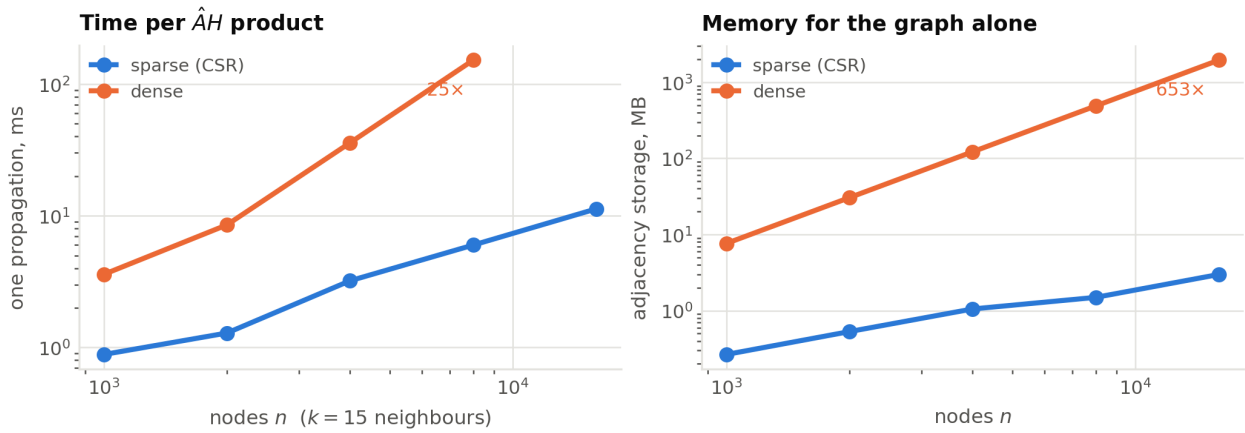


Figure 4: Measured on this machine: one $\hat{A}H$ product and the memory the adjacency occupies, for a k NN graph with $k = 15$.

The asymptotics are $O(|\mathcal{E}|d)$ against $O(n^2d)$, and the constants matter more than the exponents at the sizes single-cell work actually uses. At $n = 8000$ the sparse product is $26\times$ faster; at $n = 16,000$ the dense adjacency alone would need 1.95 GB against 3 MB, a factor of 653. The dense curve stops at 8000 because allocating the next one was not sensible on this machine, which is itself the point.

Two practical readings. First, for $n \lesssim 3000$ — a single Visium section, a patient cohort — dense is fast enough and far easier to debug; do not reach for sparse tensors before you need them. Second, the crossover is driven by $|\mathcal{E}|/n^2$, so a spatial graph at radius r large enough to give average degree 50 is a very different object from a k NN graph at $k = 15$, and the memory arithmetic should be redone rather than assumed.

1.8 The library equivalents

Once the mechanism is clear, use the library. The translation is direct:

scmgnn	PyTorch Geometric	DGL
GCNLayer(d_in, d_out)	GCNConv(d_in, d_out, normalize=False)	GraphConv(d_in, d_out, norm='none')
GATLayer(..., v2=True)	GATv2Conv(d_in, d_out, heads)	GATv2Conv(...)
GATLayer(..., v2=False)	GATConv(d_in, d_out, heads)	GATConv(...)
GINLayer(d_in, d_h, d_out)	GINConv(nn=MLP, train_eps=True)	GINConv(apply_func=MLP)
SAGELayer(d_in, d_out)	SAGEConv(d_in, d_out, aggr='mean')	SAGEConv(..., 'mean')
APPNPProp(K, alpha)	APPNP(K, alpha)	APPNPConv(k, alpha)
scatter_sum(src, idx, n)	torch_scatter.scatter_add	dgl.ops.copy_e_sum
segment_softmax(e, idx, n)	torch_geometric.utils.softmax	dgl.ops.edge_softmax

Two differences to keep in mind when swapping. PyG’s GCNConv normalises the adjacency inside forward by default — pass `normalize=False` when the graph is fixed and pre-normalised, or you pay for it every epoch. And PyG’s `edge_index` convention is `[source, target]` with messages flowing `source → target`, which matches this package; DGL’s `(u, v)` is the same direction but its `edge_softmax` normalises over *incoming* edges of `v`, so a naive port that swaps the rows will silently normalise the wrong way.

1.9 Heterogeneous and bipartite graphs

A cell–feature graph has two node types and at least two edge types, so a single weight matrix is the wrong object. The relational form of [T §B.5] is

$$h_v^{(\ell)} = \sigma \left(W_0 h_v^{(\ell-1)} + \sum_{r \in \mathcal{R}} \sum_{u \in \mathcal{N}_r(v)} \frac{1}{c_{v,r}} W_r h_u^{(\ell-1)} \right), \quad W_r = \sum_{b=1}^B a_{rb} V_b. \quad (7)$$

There are two ways to implement it, and the choice is about how many relations there are.

Few relations — one homogeneous pass per relation. Below about ten edge types this is simplest, fastest, and easiest to debug:

```
class RGCNLayer(nn.Module):
    """PyG: RGCNConv(d_in, d_out, num_relations, num_bases)"""
    def __init__(self, d_in, d_out, n_relations, n_bases=None):
        super().__init__()
        self.self_lin = nn.Linear(d_in, d_out)
        if n_bases is None:
            # one matrix per relation
            self.W = nn.Parameter(torch.empty(n_relations, d_in, d_out))
            self.comp = None
        else:
            # basis decomposition
            self.W = nn.Parameter(torch.empty(n_bases, d_in, d_out))
```

```

        self.comp = nn.Parameter(torch.empty(n_relations, n_bases))
        nn.init.xavier_uniform_(self.comp)
        nn.init.xavier_uniform_(self.W)

    def forward(self, H, edge_index, edge_type, n_relations):
        W = self.W if self.comp is None else torch.einsum("rb,bio->rio", self.comp, self.W)
        out = self.self_lin(H)
        src, dst = edge_index
        for r in range(n_relations):
            # one sparse pass per relation
            m = edge_type == r
            if not m.any():
                continue
            msg = (H[src[m]] @ W[r])
            deg = scatter_sum(torch.ones(m.sum(), device=H.device), dst[m], H.size(0))
            out = out + scatter_sum(msg, dst[m], H.size(0)) / deg.clamp(min=1).unsqueeze(-1)
        return out

```

Basis decomposition is what keeps this tractable: with $|\mathcal{R}|$ relations and $B \ll |\mathcal{R}|$ shared bases the parameter count grows in B , not $|\mathcal{R}|$. For a single-cell heterogeneous graph with a dozen edge types, $B = 4$ is usually plenty.

Bipartite graphs — do not use a relational layer at all. A cell–feature graph has exactly one edge type and two node types with *different feature dimensions*, which is a simpler object:

```

def bipartite_propagate(H_cell, H_feat, edge_index, n_cell, n_feat, W_cf, W_fc):
    """Two half-steps: cells <- features, then features <- cells."""
    c_idx, f_idx = edge_index
    # cell index, feature index
    cell_new = scatter_sum((H_feat @ W_fc)[f_idx], c_idx, n_cell) # (n_cell, d)
    feat_new = scatter_sum((H_cell @ W_cf)[c_idx], f_idx, n_feat) # (n_feat, d)
    return cell_new, feat_new

```

Note that `edge_index` here is *not* square-indexed: row 0 indexes into $[0, n_{\text{cell}})$ and row 1 into $[0, n_{\text{feat}})$. Mixing this convention with the homogeneous one — where both rows index the same node set — is the most common bug in bipartite code, and it does not raise an error: it silently gathers the wrong rows whenever $n_{\text{feat}} \leq n_{\text{cell}}$. Assert the ranges:

```

assert c_idx.max() < n_cell and f_idx.max() < n_feat

```

The guidance-graph model of Part 3 sidesteps all of this by keeping the two sides in separate tensors and joining them only in the decoder’s inner product — which is why its forward has no gather at all on the cell side.

Part 2 — How the package is organised

2.1 Four tiers, one direction of dependency

a How the package composes: four layers of abstraction, one direction of dependency

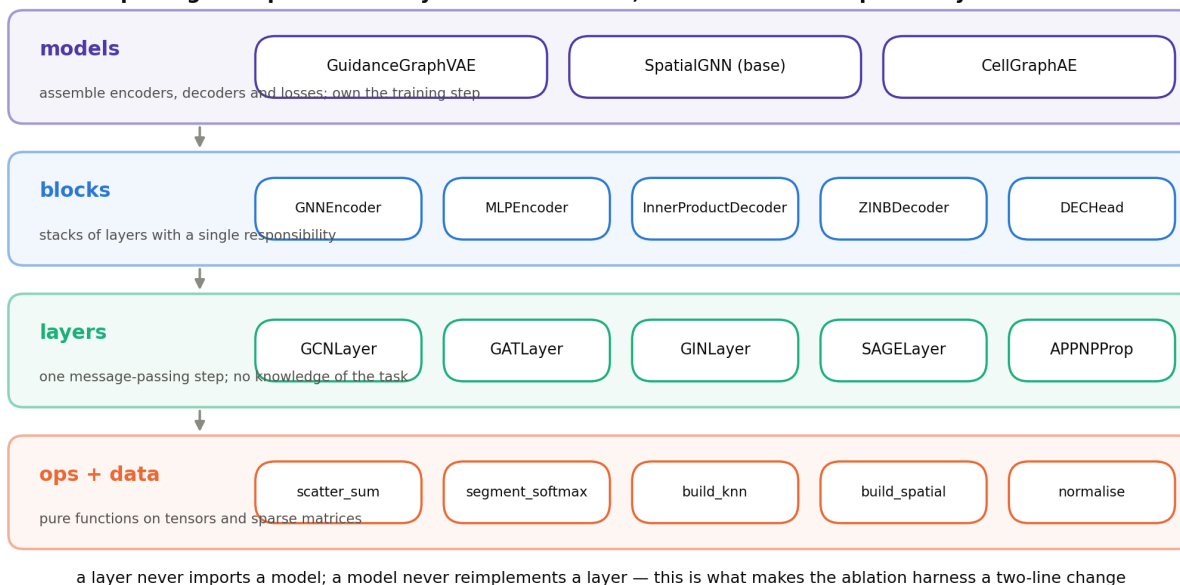


Figure 5: Each tier may import from the tier below it and never from the tier above.

The rule is one sentence: **a layer never imports a model, and a model never reimplements a layer**. Its payoff shows up in Part 6, where swapping the graph for a permuted copy is a two-line change rather than a fork of the training script, and in Part 4, where four published spatial methods become four subclasses that differ in three methods each.

The tiers are worth naming precisely because the boundary is what gets violated first:

- **ops** — pure functions on tensors: `scatter_sum`, `segment_softmax`, `to_torch_sparse`, `dirichlet_energy`. No `nn.Module`, no state.
- **graphs** — pure functions on scipy sparse matrices: `build_knn`, `build_spatial`, `build_guidance`, `normalise`, `permute_graph`. Deliberately not tensors: a graph should be inspectable, saveable and permutable before anything is allocated on a device.
- **layers** / **blocks** — one message-passing step; then stacks of them with a single responsibility (`GNNEncoder`, `ZINBDecoder`, `DECHead`, `Discriminator`).
- **models** — assemble blocks, own the loss, expose `loss(batch, graph, epoch) -> (scalar, terms, embedding)`.

That last signature is the contract the whole package is built around. `train.py` knows nothing about any model except that it satisfies it, which is what lets one loop serve a guidance-graph VAE and a spatial autoencoder.

2.2 The layout

a Repository layout

<pre>scmgnn/ — ops.py — graphs.py — layers.py — blocks.py — losses.py — models/ — cellgraph.py — guidance.py — spatial.py — train.py — ablate.py verify/ — ref.py — run_checks.py configs/ notebooks/</pre>	<pre>the package scatter_sum, segment_softmax, normalise build_knn, build_spatial, build_guidance, permute_graph GCNLayer, GATLayer, GINLayer, SAGELayer, APPNPProp GNNEncoder, ZINBDecoder, InnerProductDecoder, DECHad, GRL nb_nll, zinb_nll, gaussian_kl, dgi_loss, info_nce, cox_ph the §D.1 template GLUE-style unpaired integration SpatialGNN base + SpaGCN/STAGATE/GraphST/SEDR one loop, with the correctness assertions built in real / permuted / random / no graph, n seeds each numpy reference semantics; runs without torch every layer, re-implemented in ~200 lines of numpy the 17 checks quoted in this companion one YAML per experiment; the only place numbers live narrative only — nothing importable lives here</pre>
---	---

the rule that keeps it honest: nothing in scmgnn/ imports from notebooks/ or configs/

Figure 6: The repository, and the one rule that keeps it honest.

Two conventions are worth copying even if you take nothing else.

verify/ is a parallel implementation, not a test of the main one. It re-implements every layer in about two hundred lines of numpy, and the checks compare the two. A test that calls the code under test and asserts it equals itself proves nothing; a second implementation, written from the equations rather than from the first implementation, catches real errors — as it did for GATv2 in §1.4. It also runs without PyTorch, which means the numbers in this document can be regenerated on any machine.

configs/ is the only place numbers live. Learning rates, k , radii, loss weights. A hyperparameter hard-coded in a model class is a hyperparameter that will not appear in the paper’s methods section.

2.3 Configuration objects, not keyword arguments

```
@dataclass
class CellGraphConfig:
    d_hidden: int = 128
    d_latent: int = 32
    n_clusters: int = 10
    kind: str = "gcn"           # 'gcn' | 'gat' | 'appnp'
    likelihood: str = "zinb"    # 'zinb' | 'nb' | 'mse'
    lam_graph: float = 0.0      # 0 disables the GAE term
    lam_dec: float = 0.0        # 0 disables self-training
    lam_kl: float = 1.0
    variational: bool = True
```



```
dropout: float = 0.1
kl_warmup: int = 30
```

A dataclass rather than `**kwargs` for three reasons that are all about reproducibility: it serialises to the run directory with `asdict`, so an experiment is reconstructable from its output; it fails loudly on a typo where a `kwargs` dict fails silently; and it makes the ablation grid a list of dataclasses rather than a nest of dictionaries.

Note what the defaults encode. `lam_graph = 0.0` and `lam_dec = 0.0` mean the template of [T §D.1] starts as a plain count VAE on a graph — the extra terms are opt-in, and each one should have to earn its weight against an ablation rather than arriving switched on because the paper being reimplemented had it.

Part 3 — Guidance graphs, implemented

The hardest integration problem is unpaired multi-omics: RNA on one set of cells, ATAC on another, no shared cells and no shared features. The guidance-graph construction of [T §D.2] solves it by refusing to match cells at all and embedding *features* instead, through a graph that encodes prior biology.

3.1 The objective, restated for implementation

With modalities indexed by k , cells c , features j , cell embeddings $u_c \in \mathbb{R}^d$ and feature embeddings $v_j \in \mathbb{R}^d$:

$$\rho_{cj} = \text{softmax}_j(u_c^\top v_j + b_j), \quad x_{cj} \sim \text{NB}(\ell_c \rho_{cj}, \theta_j), \quad (8)$$

$$\mathcal{L} = \underbrace{\sum_k \text{NB-NLL}_k + \beta \text{KL}_k}_{\text{per-modality ELBO}} + \lambda_{\mathcal{G}} \underbrace{\mathcal{L}_{\mathcal{G}}(v)}_{\text{graph ELBO}} + \lambda_D \underbrace{\mathcal{L}_D(u)}_{\text{adversarial}}. \quad (9)$$

Four things follow directly from these two equations, and each is a design decision in code.

The softmax in (8) is **over features within a modality**, so each modality slices its own block out of v — hence the offset bookkeeping in `self.offsets`. The library size ℓ_c multiplies the rate rather than being learned, which is what makes the model depth-aware without a normalisation step. The dispersion θ_j is per feature, not per cell, so it is a bare `nn.Parameter` and not a decoder output. And the adversarial term enters with a **plus** sign because the minus lives inside the gradient-reversal layer (§3.4) — writing it with a minus here and also using a GRL is a sign error that produces an encoder cheerfully helping the discriminator.

3.2 Building the guidance graph

The graph is prior annotation, not data. Nodes are all features of all modalities concatenated; edges are peak→gene links from a genome annotation, signed by whether the element is expected to activate or repress:

```
from scmgnn.graphs import build_guidance, normalise
from scmgnn.ops import to_torch_sparse, to_edge_index

# pairs: (gene_index, peak_index) for every peak in a gene body or promoter window
A_g = build_guidance(pairs, n_genes=len(genes), n_peaks=len(peaks), signs=signs)
G_hat = to_torch_sparse(normalise(A_g))          # for the graph encoder
g_edges, g_sign = to_edge_index(A_g)            # for the reconstruction term
```

`build_guidance` returns a symmetric $(g_1 + g_2)^2$ matrix with self-loops. Its size is the number of

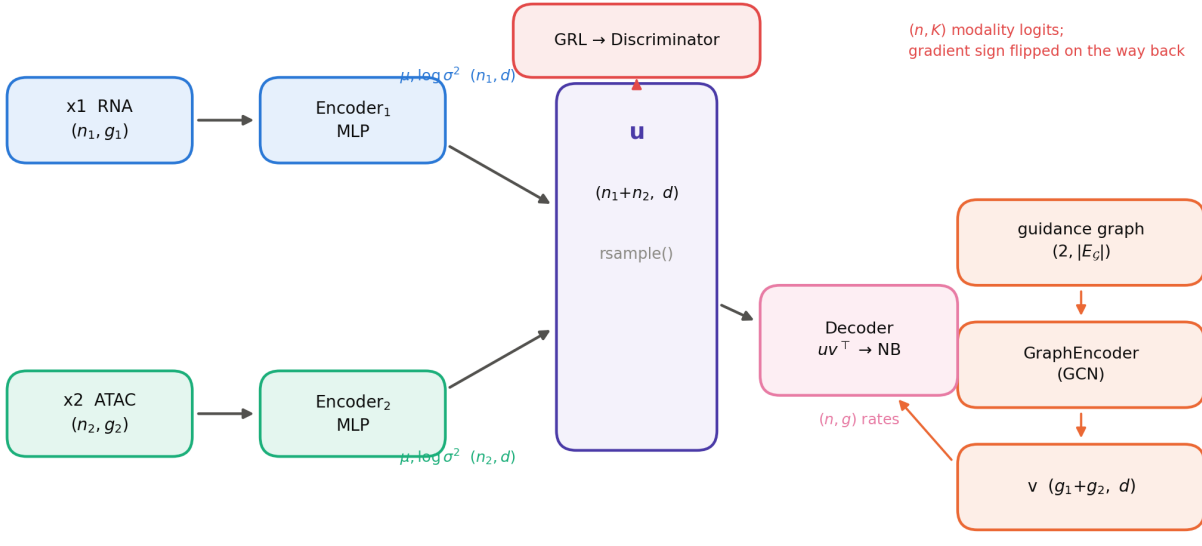
features, not cells, so it costs the same whether the experiment has ten thousand cells or ten million — the structural reason this approach scales where cell-matching does not.

The signs are the part most implementations drop, and they change the loss. A repressive edge is a negative example of co-embedding, so its target is 0:

```
def signed_graph_bce(z, edge_index, sign, num_nodes, neg_ratio=1):
    src, dst = edge_index
    logits = (z[src] * z[dst]).sum(-1)
    target = (sign > 0).float()          # -1 edges are negatives, not absences
    loss = F.binary_cross_entropy_with_logits(logits, target)
    ...
```

3.3 The wiring

a Guidance-graph model: every tensor that crosses a module boundary, with its shape



b loss terms $\text{nb_nll}(x, \mu, \theta)$ $\text{kl}(\mu, \text{logvar})$ $\text{graph_bce}(v, \text{edges})$ $-\lambda \cdot \text{ce}(\text{disc}(u), \text{modality})$

four scalars, summed; the minus sign is inside the GRL, not the loss

Figure 7: Every tensor that crosses a module boundary in the guidance-graph model, with its shape.

```
class GuidanceGraphVAE(nn.Module):
    def __init__(self, cfg, n_batches=None):
        super().__init__()
        self.encoders = nn.ModuleDict({
            m: MLPDecoder(g, cfg.d_hidden, cfg.d_latent,  # one per modality,
                          n_cov=n_batches[m], dropout=cfg.dropout)  # all into ONE space
        })
        for m, g in cfg.feature_dims.items():
            self.log_theta = nn.ParameterDict({
                m: nn.Parameter(torch.zeros(g))  # per-feature dispersion
            })
        self.decoders = nn.ModuleDict({
```

```

        m: InnerProductDecoder(g) for m, g in cfg.feature_dims.items())

self.n_features = sum(cfg.feature_dims.values())
self.feature_emb = nn.Parameter(torch.randn(self.n_features,
                                             cfg.graph_hidden) * 0.01)
self.graph_encoder = GNNEncoder(cfg.graph_hidden, cfg.graph_hidden,
                                cfg.d_latent, kind="gcn", variational=True)
self.discriminator = Discriminator(cfg.d_latent, len(self.modalities))

self.offsets, off = {}, 0 # where each modality's block of v is
for m, g in cfg.feature_dims.items():
    self.offsets[m] = (off, off + g); off += g

```

The feature nodes have no natural input features — a peak is not a vector — so `feature_emb` is a free embedding table, initialised small, that the graph encoder then smooths over the prior edges. This is the mechanism by which a gene and its linked peaks end up near each other in v , and therefore the mechanism by which a cell expressing the gene and a cell with the peak open end up near each other in u .

The forward pass is short because the structure does the work:

```

def forward(self, batch, guidance_A_hat):
    v, mu_v, logvar_v = self.encode_features(guidance_A_hat) # (n_feat, d)
    out = {}
    for m, b in batch.items():
        u, mu_u, logvar_u = self.encode_cells(b["x"], m, b.get("cov"))
        lo, hi = self.offsets[m]
        rho = self.decoders[m](u, v[lo:hi]) # (n_m, g_m)
        out[m] = {"u": u, "mu": mu_u, "logvar": logvar_u,
                  "rho": rho, "theta": self.log_theta[m].exp().clamp(1e-4, 1e4)}
    return out, (v, mu_v, logvar_v)

```

3.4 The adversarial term, in one backward pass

$$\min_{\phi} \max_{\omega} \mathbb{E} [\log D_{\omega}(u)] \iff \text{one loss, with } \frac{\partial \mathcal{L}_D}{\partial u} \text{ multiplied by } -\lambda. \quad (10)$$

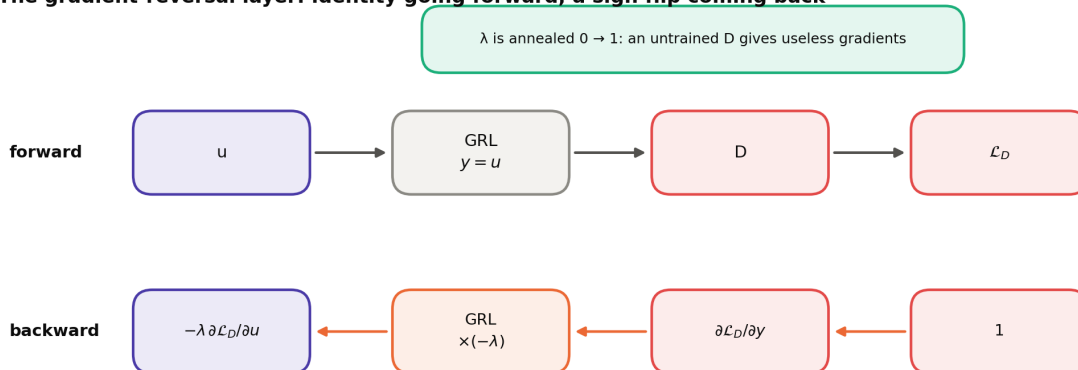
```

class GradientReversal(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, lambd):
        ctx.lambd = lambd
        return x.view_as(x)

    @staticmethod
    def backward(ctx, grad):
        return -ctx.lambd * grad, None

```

a The gradient-reversal layer: identity going forward, a sign flip coming back



the encoder therefore maximises the discriminator's loss while the discriminator minimises it — one optimiser, one backward pass, no alternating updates

Figure 8: Gradient reversal: identity forward, a sign flip backward.

That is the whole of adversarial alignment. No alternating updates, no separate optimiser, no `detach()` gymnastics: the discriminator minimises its cross-entropy while the encoder — receiving the negated gradient — maximises it, in a single backward pass.

```
lam = cfg.lam_adv * min(1.0, epoch / max(cfg.adv_warmup, 1))
u_all = grad_reverse(torch.cat(u_all), lam)
terms["adv"] = F.cross_entropy(self.discriminator(u_all), torch.cat(y_all))
```

The warm-up is not decoration. At initialisation the discriminator is random, its gradients are noise, and feeding that noise into the encoder at full strength reliably destroys the latent structure in the first few epochs. Anneal λ from 0 over the first 50 epochs and the failure disappears.

The failure mode to watch. Nothing in (10) requires that a T cell land on a T cell; it requires only that the two clouds overlap. When cell-type composition differs strongly between modalities, an adversarial integrator will happily superimpose populations that share no biology, because that is the cheapest way to fool D . Symptoms are a discriminator accuracy pinned at chance from epoch 20 together with a *worse* biological-conservation score. The diagnostic is to hold out a shared cell type and check it still co-clusters.

3.5 Reading the biology back out

The guidance graph is a prior, so the posterior over its edges is a result — this is the regulatory inference that comes free with the architecture:

```
@torch.no_grad()
def regulatory_scores(self, guidance_A_hat, gene_idx, peak_idx):
    self.eval()
    v, _, _ = self.encode_features(guidance_A_hat)
    return torch.sigmoid((v[gene_idx] * v[peak_idx]).sum(-1))
```

Two cautions on using it. The scores are shaped by the prior that produced the graph, so a highly-

scored peak–gene pair that was an input edge is not evidence; the interesting output is the *change* in score relative to the prior weight, and pairs that were not edges at all. And like every explanation in [T §B.10], the ranking moves under reinitialisation — aggregate over seeds before calling anything a finding.

3.6 What goes wrong, and what it looks like

Symptom	Cause	Fix
loss falls, embeddings are noise	posterior collapse	lengthen KL warm-up; check $\text{kl} > 1\text{e-}3$
latent splits cleanly by modality	λ_{adv} too small, or GRL sign wrong	check the gradient sign test in tests/
modalities overlap, cell types do not	adversarial term dominating	reduce λ_{adv} ; hold out a shared type
NaN after a few hundred steps	softmax over-/underflow in the decoder	clamp <code>log_theta</code> ; check library sizes are non-zero
graph term does nothing	lam_graph too small relative to $ \mathcal{E}_{\mathcal{G}} $	it is summed, not averaged — scale by $1/ \mathcal{E}_{\mathcal{G}} $

The last row is the one that costs the most time. Reconstruction is averaged over cells, the graph term is summed over edges, and a guidance graph with 10^6 edges will contribute a term four orders of magnitude larger than the ELBO unless it is normalised. Whenever a loss weight needs to be 10^{-6} to work, the real problem is that two terms are reduced differently.

Part 4 — Spatial GNN variants, implemented

Spatial data is where graph methods have their cleanest justification: the edges come from coordinates, not from the expression being fitted. [RL §4.1] contains twenty-two methods for one task. Four of them cover the design space, and implementing all four against one base class makes the comparison honest.

4.1 The graph, and its units

```
def build_spatial(coords, mode="radius", r=None, k=6, l=None):
    if mode == "radius":
        A = radius_neighbors_graph(coords, radius=r, mode="distance")
    elif mode == "knn":
        A = kneighbors_graph(coords, n_neighbors=k, mode="distance")
    elif mode == "deLaunay":
        ... # contact adjacency, no parameter
    A = A.maximum(A.T).tocsr()
    if l is not None:
        A.data = np.exp(-(A.data ** 2) / (2.0 * l ** 2)) # Gaussian on distance
    else:
        A.data = np.ones_like(A.data)
    A.setdiag(0.0); A.eliminate_zeros()
    return A
```

The single most important line in a spatial pipeline is the one that sets r , and it is meaningless without units. Visium spots are $100\text{ }\mu\text{m}$ across on a $100\text{ }\mu\text{m}$ pitch; Xenium cells are $10\text{--}20\text{ }\mu\text{m}$. A radius of 50 models contact at Xenium resolution and nothing at all at Visium resolution. `coords` must be in microns before it reaches this function, and the value of r belongs in the methods section:

```
report = graph_report(build_spatial(coords_um, mode="radius", r=150.0))
# choose r so that deg_mean lands in 4-8 for Visium, 6-12 for imaging data
```

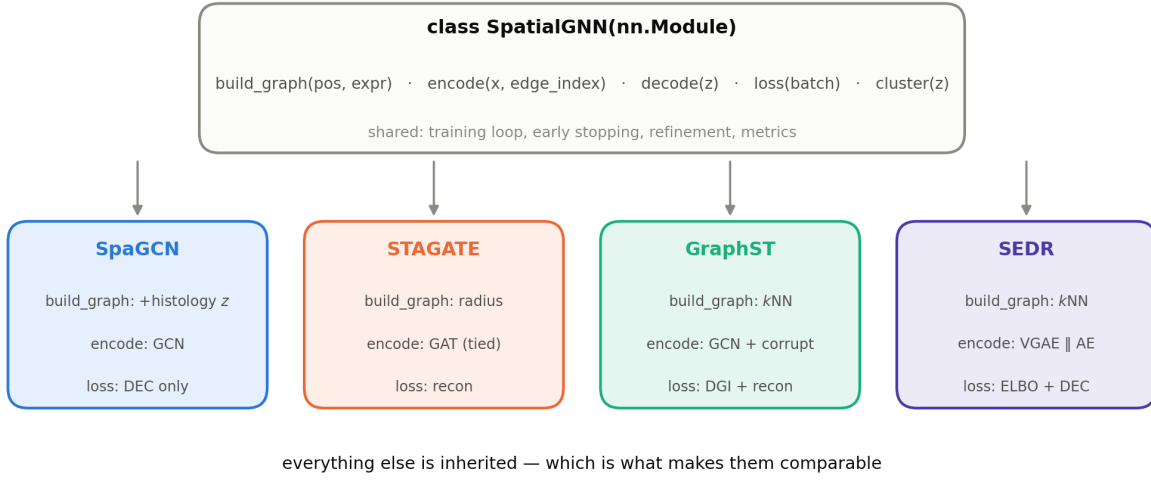
`mode="knn"` fixes degree and lets physical scale float, which is the right default for imaging data with strongly varying density; `mode="deLaunay"` is parameter-free and is what a cell–cell contact model should use.

4.2 Four methods, three overridden hooks

```
class SpatialGNN(nn.Module):
    """Base. Subclasses override build_graph / encode / loss and nothing else."""
    kind = "gcn"

    def build_graph(self, coords, expr=None, image_stat=None): ...
    def encode(self, x, graph): return self.encoder(x, graph)
```

a Four published spatial methods as four subclasses of one base



If a fair comparison needs more than three overridden methods, the difference between the papers is not the architecture

Figure 9: The base class owns everything the papers share. Each subclass overrides at most three methods.

```
def loss(self, x, graph, epoch=0): ...
# shared: _prepare, cluster, refine, and the training step from train.py
```

SpaGCN — put everything in the graph. Histology enters as a third coordinate, then one Gaussian kernel over all three axes:

$$w_{ij} = \exp\left(-\frac{d_{ij}^2}{2l^2}\right), \quad d_{ij}^2 = (x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2. \quad (11)$$

```
def build_spagcn_graph(coords, image_stat, l, scale=1.0):
    z = (image_stat - image_stat.mean()) / (image_stat.std() + 1e-12)
    z = z * scale * np.std(coords)          # put histology on the scale of geometry
    P = np.column_stack([coords, z])
    W = np.exp(-((P[:, None, :] - P[None, :, :]) ** 2).sum(-1) / (2.0 * l ** 2))
    np.fill_diagonal(W, 0.0)
    return sp.csr_matrix(W)
```

The rescaling line is the whole method. `image_stat` is a per-spot scalar from the H&E patch; standardising it and multiplying by the spread of the coordinates is what makes `scale` interpretable as “how loud is histology relative to geometry”. Note that this builds a **dense** $n \times n$ kernel — fine for one Visium section at $n \approx 4000$ (§1.7), not fine at $n = 10^5$, where the kernel has to be thresholded or restricted to a neighbour list first.

STAGATE — learn the edge weights. The graph stays binary and geometric; attention decides what to trust:


```

class STAGATE(SpatialGNN):
    kind = "gat"

    def build_graph(self, coords, expr=None, image_stat=None):
        A = build_spatial(coords, mode="radius" if self.cfg.radius else "knn",
                          r=self.cfg.radius, k=self.cfg.k)
        if self.prune_labels is not None:      # hard version of what attention does
            src, dst = A.nonzero()
            keep = self.prune_labels[src] == self.prune_labels[dst]
            A = sp.csr_matrix((np.ones(keep.sum()), (src[keep], dst[keep])), A.shape)
        return self._prepare(A)

    def loss(self, x, graph, epoch=0):
        z = self.encode(x, graph)
        recon = F.mse_loss(self.decoder(z), x)
        return recon, {"recon": float(recon)}, z

```

The optional pruning step deserves a comment because it is often presented as a refinement and is in fact an admission: it deletes edges between spots that a pre-clustering assigned to different domains, which is exactly what the attention was supposed to learn to do softly. On low-depth sections the learned α often comes out near-uniform, at which point STAGATE is SpaGCN with more parameters — and the diagnostic is one line, `alpha.std()`, logged during training.

GraphST — self-supervision instead of reconstruction. The corruption is the design decision:

```

class GraphST(SpatialGNN):
    def loss(self, x, graph, epoch=0):
        z = self.encode(x, graph)
        perm = torch.randperm(x.size(0), device=x.device)    # shuffle FEATURES,
        z_corrupt = self.encode(x[perm], graph)              # keep the graph
        ssl = dgi_loss(z, z_corrupt, self.W_dgi)
        recon = F.mse_loss(self.decoder(z), x)
        return ssl + recon, {"dgi": float(ssl), "recon": float(recon)}, z

```

$$\mathcal{L}_{\text{DGI}} = -\frac{1}{2} \left(\log \sigma(h_i^\top W g) + \log (1 - \sigma(\tilde{h}_i^\top W g)) \right), \quad g = \sigma\left(\frac{1}{n} \sum_i h_i\right). \quad (12)$$

Permuting the rows of x while keeping `edge_index` fixed destroys the correspondence between a spot's expression and its neighbourhood, and preserves the degree distribution exactly. Permuting the *graph* instead would be a different and much weaker corruption — the model could then solve the task from degree alone.

SEDR — keep a non-graph channel. Two encoders, concatenated:

```

class SEDR(SpatialGNN):
    def encode(self, x, graph):
        mu, logvar = self.encoder(x, graph)                # variational graph AE

```

```

z_g = reparameterise(mu, logvar) if self.training else mu
self._kl, self._mu_g = gaussian_kl(mu, logvar), mu
return torch.cat([self.expr_enc(x), z_g], dim=-1)    # (n, 2d)

```

Redundant by design: if the spatial graph is uninformative — a tissue with no spatial organisation, coordinates with registration error — the expression channel still carries the signal. That is why SEDR is the safe default on unfamiliar data and why its ceiling on well-structured tissue is lower than the attention-based methods’.

4.3 Refinement, and the metric it flatters

Every method in this family ends with a majority vote over spatial neighbours:

```

@torch.no_grad()
def refine(self, labels, coords, k=6):
    A = build_spatial(coords, mode="knn", k=k).tocsr()
    out = labels.copy()
    for i in range(len(labels)):
        nb = A.indices[A.indptr[i]:A.indptr[i + 1]]
        vals, counts = np.unique(labels[nb], return_counts=True)
        if counts.max() > len(nb) / 2:
            out[i] = vals[counts.argmax()]
    return out

```

Six lines, worth several ARI points on their own, and applied by every method in [RL §4.1] — sometimes without being mentioned in the ablation table. Since the standard spatial metrics reward spatial coherence, and refinement manufactures spatial coherence directly, a comparison that refines the proposed method and not the baselines is measuring the smoother. `scmgnn` therefore returns both, and the training script reports both:

```

labels_raw      = model.cluster(z, coords=None)
labels_refined  = model.refine(labels_raw, coords)

```

4.4 Putting one together

```

from scmgnn.models.spatial import REGISTRY, SpatialConfig
from scmgnn.train import train, TrainConfig

cfg = SpatialConfig(d_latent=32, n_clusters=7, radius=150.0, k=6, l=1.0)
model = REGISTRY["stagate"](n_genes=adata.n_vars, cfg=cfg)
graph = model.build_graph(adata.obsm["spatial"], expr=X)    # returns A_hat or edges

model, history = train(model, {"x": X}, graph,
                        TrainConfig(epochs=600, lr=1e-3, dec_init_epoch=200))

```

```
_, _, z = model.loss({"x": X}, graph, epoch=0)
labels = model.cluster(z, coords=adata.obsm["spatial"])
```

`dec_init_epoch` is the one ordering constraint that matters: the DEC head must be k-means-initialised on an embedding that has already learned something. Initialise it at epoch 0 and the self-training objective spends the rest of the run confirming the structure of random noise — a failure that looks like fast convergence to a confidently wrong answer.

Part 5 — Training

5.1 One loop, with the failures asserted away

Every model in the package satisfies one contract — `loss(batch, graph, epoch) -> (scalar, terms, embedding)` — so there is one loop:

```
def train(model, batch, graph, cfg, loss_kwargs=None, verbose=True):
    set_seed(cfg.seed)
    opt = torch.optim.Adam(model.parameters(), lr=cfg.lr,
                           weight_decay=cfg.weight_decay)
    sched = torch.optim.lr_scheduler.ReduceLROnPlateau(opt, factor=0.5, patience=15)
    best, wait = math.inf, 0

    for epoch in range(cfg.epochs):
        if cfg.dec_init_epoch is not None and epoch == cfg.dec_init_epoch:
            model.eval()
            with torch.no_grad():
                _, _, z = model.loss(batch, graph, epoch=epoch)
                model.dec_head.initialise(z)          # k-means on a trained embedding

        model.train()
        opt.zero_grad(set_to_none=True)
        loss, terms, z = model.loss(batch, graph, epoch=epoch)
        if not torch.isfinite(loss):
            raise FloatingPointError(f"non-finite loss at epoch {epoch}: {terms}")
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), cfg.grad_clip)
        opt.step()
        sched.step(loss.item())

        if epoch % cfg.log_every == 0:
            gnorm = _grad_norm(model)
            if cfg.assert_kl and "kl" in terms:
                assert terms["kl"] > 1e-3, (
                    "posterior collapse: KL is ~0, the latent is being ignored.")
            assert gnorm > 0, "no gradient reached the parameters"
        ...
```

The two assertions are the reason this loop is worth showing. Both failures they catch are **silent**: the loss falls, training completes, and the embedding is garbage.

Posterior collapse ($kl \rightarrow 0$) means the decoder learned to reconstruct without the latent, so z is prior noise. In a graph model this is routinely misread as “the graph did not help”.

A *zero gradient norm* means an entire branch is disconnected — most often a tensor that was `.detach()`ed during debugging and never reconnected, or a loss term multiplied by a weight that is exactly zero.

A third check belongs in the same place but is too slow to run every epoch, so it is a separate function:

```
@torch.no_grad()
def smoothing_probe(model, batch, A_hat, depths=(1, 2, 4, 8, 16)):
    """Dirichlet energy after extra propagation steps: where does this graph collapse?"""
    _, _, z = model.loss(batch, A_hat, epoch=0)
    out, h, prev = {}, z, 0
    for d in depths:
        for _ in range(d - prev):
            h = torch.sparse.mm(A_hat, h)
            prev, out[d] = d, dirichlet_energy(h, A_hat)
    return out
```

Run it once per dataset. The depth at which the energy falls off a cliff is the depth at which the model stops distinguishing cells, and it depends on the graph — a well-clustered k NN graph tolerates far more propagation than a dense spatial one, exactly as the spectral-gap argument in [T §B.6] predicts.

5.2 The ordering constraints

Three things must happen in the right order, and each has cost a lot of people a lot of time:

KL warm-up before anything else. β annealed from 0 to 1 over the first 10–20% of epochs. Start at $\beta = 1$ and a strong decoder collapses the posterior in the first fifty steps.

DEC initialisation after the encoder has learned something. `dec_init_epoch` around half of the planned run. Self-training amplifies whatever structure exists when it starts.

Adversarial warm-up alongside. λ_{adv} from 0 (§3.4). An untrained discriminator emits noise, and the GRL feeds that noise straight into the encoder.

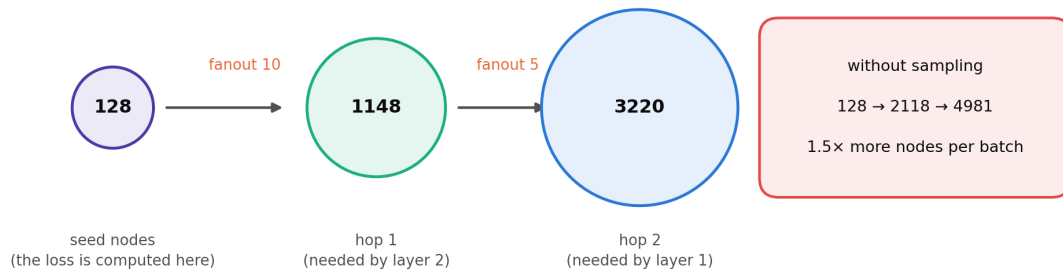
All three are the same principle: a self-referential objective — one whose target depends on the model’s own current output — must not be switched on before the model has an output worth referring to.

5.3 Minibatching, and when it actually helps

```
from torch_geometric.loader import NeighborLoader
loader = NeighborLoader(data, num_neighbors=[10, 5], batch_size=128, shuffle=True)
for sub in loader:
    loss, terms, z = model.loss({"x": sub.x}, sub.edge_index, epoch=epoch)
    # the loss is computed on sub.batch_size seed nodes only:
    loss = loss[:sub.batch_size].mean()
```

That last line is the one people get wrong. A sampled subgraph contains seed nodes *and* their

a Neighbour sampling: what a two-layer minibatch actually loads



measured, $n = 5000$, $k = 15$, 128 seeds: the unsampled 2-hop neighbourhood reaches 100% of the dataset — sampling caps it at $Bs_1s_2 = 6400$, here 3220

the $O(1)$ -in- n promise only pays off once $n \gg Bs^L$; on a small dense k NN graph sampling buys little and biases the gradient

Figure 10: Measured: the two-hop neighbourhood of a 128-cell batch on a k NN graph with $n = 5000$, $k = 15$.

sampled neighbourhoods; the neighbourhood nodes exist to provide messages, and their own representations are incomplete because their neighbours were not loaded. Computing the loss over all nodes in the block trains on representations the model would never produce at inference.

The measured numbers in Figure 10 are worth internalising. The unsampled two-hop neighbourhood of a 128-cell batch reaches 4981 of 5000 nodes — essentially the entire dataset. Sampling with fanouts (10, 5) brings it to 3220, bounded above by $Bs_1s_2 = 6400$. So the $O(1)$ -in- n promise is real but only pays off once $n \gg Bs^L$: on a graph of a few thousand cells, neighbour sampling costs you a biased gradient and buys almost nothing. Train full-batch until the graph does not fit, and only then reach for the loader.

For the case where it genuinely does not fit, subgraph batching is usually the better fit for single-cell data — with one caveat that is easy to miss. A k NN cell graph partitions almost perfectly along cell types, so METIS parts correlate with the labels, and each batch’s gradient is systematically biased towards one cell type. Randomising which parts are combined per batch is not optional.

5.4 Determinism

```
def set_seed(seed):
    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.use_deterministic_algorithms(True, warn_only=True)
```

`warn_only=True` rather than hard failure, because `scatter_add` on CUDA has no deterministic implementation: floating-point addition is not associative and the order in which edges land in a bucket varies between runs. This means **a GNN is not bit-reproducible on GPU even with a fixed seed**, and the run-to-run spread that produces is real and often comparable to the difference between published methods.

The consequence for reporting: five seeds minimum, and report mean and spread. A single number per dataset is a draw from a distribution presented as a measurement.

5.5 Where the time actually goes

```
with torch.profiler.profile(
    activities=[torch.profiler.ProfilerActivity.CPU],
    record_shapes=True) as prof:
    for _ in range(10):
        loss, _, _ = model.loss(batch, graph, epoch=0)
        loss.backward()
print(prof.key_averages().table(sort_by="self_cpu_time_total", row_limit=8))
```

On a typical single-cell run the ranking is stable and not what people expect: the dense Linear on $g \approx 2000$ genes dominates, the sparse propagation is cheap (§1.7), and for attention models the scatter and its backward pass come second. The optimisation that follows is therefore to reduce the *feature* dimension — highly variable genes, a PCA input — rather than to sample the graph. Sampling the graph is what you do when memory, not time, is the binding constraint.

Part 6 — Testing, and finding out whether the graph mattered

6.1 Four layers of test, cheapest first

a What to test in a GNN, and in what order

LAYER	WHAT IT CATCHES	COST
1. invariants	permutation equivariance · attention rows sum to 1 · likelihood integrates to 1	seconds
2. numerics	analytic gradient vs finite difference · no NaN at extreme inputs · dtype, device	seconds
3. shapes and wiring	one forward pass on 20 fake nodes, every intermediate shape asserted	seconds
4. learning	overfit 50 cells to near-zero loss · loss decreases on one real slice	minutes

a model that trains is not a model that is correct

silent failure 1 — posterior collapse
assert kl > 1e-3 every logging step

silent failure 2 — the graph is decoration
train on a degree-preserving rewiring and compare

Figure 11: What to test, in the order that finds bugs fastest.

The ordering is not arbitrary: each layer costs an order of magnitude more than the one above it and catches a class of bug the one above cannot.

Invariants are properties the mathematics guarantees, so a violation is unambiguously a bug. Permutation equivariance; attention rows summing to one; a likelihood integrating to one over its support. These take seconds and catch what no loss curve reveals.

Numerics — analytic gradients against finite differences, no NaN at extreme inputs, dtype and device consistency.

Shapes and wiring — one forward pass on twenty fake nodes with every intermediate shape asserted. Fast, and it is what makes refactoring safe.

Learning — overfit fifty cells to near-zero loss. If a model cannot memorise fifty cells, no hyperparameter will save it on fifty thousand. This is the only layer that needs minutes rather than seconds, and it is meaningless until the three above pass.

6.2 The checks, and what they actually print

`verify/run_checks.py` runs the numpy reference implementations and prints the numbers this document quotes. It needs neither torch nor a GPU:


```
$ python3 verify/run_checks.py
PASS gcn_dense_vs_sparse          max|diff|=2.22e-16
PASS gcn_sparse_vs_scatter        max|diff|=0.00e+00
PASS permutation_equivariance     max|diff|=0.00e+00
PASS attention_sums_to_one        max|sum-1|=8.88e-16
PASS gat_static_attention         GAT pairwise-order disagreements: 0/8387
PASS gatv2_dynamic_attention      GATv2 pairwise-order disagreements: 760/8387
PASS sum_distinguishes_multisets  sum 4.0 vs 3.0; mean 1.333 vs 1.500; max 2.0 vs 2.0
PASS zinb_matches_mixture         max|diff|=0.00e+00
PASS zinb_normalised              total mass=1.0000000000
PASS zinb_sign_bug_detected       buggy total mass=1.1740
PASS nb_gradient                  max|diff|=1.49e-09
PASS dec_sharpens                  mean max q=0.530 -> p=0.638
PASS appnp_fixed_point            max|diff|=3.33e-16
PASS sampling_fanout              nodes per hop: [4, 33, 129] (of 200 total)
```

17 checks, 0 failures

Three of these lines are worth dwelling on.

zinb_normalised / zinb_sign_bug_detected. The ZINB log-likelihood has an asymmetry that is easy to get wrong:

$$\log p(x) = \begin{cases} \text{softplus}(\log p_{\text{NB}}(0) - g) - \text{softplus}(-g), & x = 0, \\ \log p_{\text{NB}}(x) - \text{softplus}(+g), & x > 0, \end{cases} \quad (13)$$

with gate logit g and $\pi = \sigma(g)$. Note $\text{softplus}(-g)$ in one branch and $\text{softplus}(+g)$ in the other. Use $-g$ in both — the natural-looking mistake — and the loss still decreases, gradients still flow, nothing warns you; the density simply no longer integrates to one (measured: 1.174) and the fitted dispersion drifts to compensate. The check is one line: `sum exp of the log-probability over  $x = 0, 1, 2, \dots$  and confirm it reaches 1.`

gat_static_attention. Zero out of 8387 is the separability of (4) confirmed empirically, not a soft signal. This test found a genuine bug in an earlier draft of this material.

sampling_fanout. Four seed nodes reach 129 of 200 in two hops — the neighbourhood explosion of §5.3 in miniature.

The PyTorch-side suite mirrors these in `tests/test_layers.py` and runs with `pytest -q`. It uses `importorskip`, so on a machine without torch it skips rather than fails and the numpy checks remain the safety net.

6.3 The ablation harness

The four graph variants of [T §E.2], as a loop:

```

VARIANTS = {
    "real":      lambda A, s: A,
    "permuted":  lambda A, s: permute_graph(A, seed=s),  # same degrees, no biology
    "random":    lambda A, s: random_graph(A, seed=s),   # same edge count
    "none":      lambda A, s: empty_graph(A),           # degenerates to an MLP
}

def ablate(build_model, batch, A, evaluate, cfg=TrainConfig(), seeds=range(5),
           variants=tuple(VARIANTS), kind="gcn"):
    results = {}
    for name in variants:
        scores = []
        for s in seeds:
            Av = VARIANTS[name](A, s)
            graph = (to_edge_index(Av)[0] if kind == "gat"
                     else to_torch_sparse(normalise(Av)))
            model, _ = train(build_model(), batch, graph,
                            TrainConfig(**{**cfg.__dict__, "seed": s}), verbose=False)
            scores.append(float(evaluate(model, batch, graph)))
        results[name] = (float(np.mean(scores)), float(np.std(scores)), scores)
    return results

```

`permute_graph` is a degree-preserving double-edge swap, so the null graph has the *same degree sequence* as the real one and none of the biology. This distinguishes two claims that are routinely conflated: “the graph carries signal” and “graph regularisation stabilises training”. Only the first is a claim about biology, and only the permuted control separates them.

6.4 Reading the result

The harness states its own conclusion, because the arithmetic is where wishful thinking enters:

```

def verdict(results, key_real="real", key_null="permuted"):
    m_r, sd_r, _ = results[key_real]
    m_n, sd_n, _ = results[key_null]
    pooled = (sd_r ** 2 + sd_n ** 2) ** 0.5
    gap = m_r - m_n
    if gap > 2 * pooled:
        return f"the graph carries signal: {gap:.3f} above the permuted control, ..."
    if gap > pooled:
        return f"weak evidence: ... add seeds before believing it"
    return "no evidence the graph matters: the permuted control is within noise"

```

```

real      0.6841 ± 0.0119
permuted  0.6702 ± 0.0143
random    0.6118 ± 0.0201
none      0.5883 ± 0.0092

```

weak evidence: 0.014 above control, only 0.8x the pooled spread –
add seeds before believing it

That illustrative table is the pattern to expect, and it is uncomfortable in a specific way: the graph clearly beats *no graph* by a wide margin, and barely beats a *permuted* graph. Read carefully, that says most of the benefit came from smoothing over a neighbourhood of the right size and degree distribution, and rather little from the neighbourhood being the biologically correct one. That is a real result about the method, and it is invisible to any comparison that only ablates the graph away entirely.

Run this before writing the paper, not after review.

Part 7 — Porting a paper

Most of the practical work in this field is not inventing an architecture; it is reading someone's method section, or their repository, and getting it to run on your data. The failure mode is silent: a reimplementaion that trains, produces plausible clusters, and differs from the original in a way nobody notices.

7.1 Read the code, not the equations

The single most reliable finding from reproducing methods in [RL §4.1] is that **released code and published equations disagree more often than not**, and where they disagree, the code is what produced the numbers in the paper. Six specific places to look before writing anything:

1. **Where normalisation happens.** Is $\hat{A}$ symmetric or random-walk? Are self-loops added before or after? Is the graph re-normalised every forward pass?
2. **What the reconstruction target is.** Raw counts, $\log p$, scaled, or PCA? A method that reconstructs `adata.X` after `sc.pp.scale` is fitting a Gaussian to z-scores, whatever the paper's likelihood says.
3. **Whether there is a refinement step.** §4.3. Often in the demo notebook, rarely in the ablation table.
4. **How many clusters are passed in.** If k is set to the number of ground-truth domains, the reported metric is an upper bound.
5. **The order of the loss terms' reductions.** Summed over cells or averaged? This decides the loss weights and is almost never stated (§3.6).
6. **What the seed does.** If the demo fixes one seed and the paper reports one number, you are looking at a draw, not a measurement.

7.2 A worked port

Suppose a paper describes “a graph attention autoencoder with an adaptive spatial graph and a self-supervised consistency term”. Working from the tiers of Part 2, that is three decisions:

```
class PaperMethod(SpatialGNN):
    kind = "gat"  # decision 1: the operator

    def build_graph(self, coords, expr=None, image_stat=None):
        # decision 2: "adaptive" -- the paper means kNN with k chosen per section
        k = int(np.clip(np.sqrt(len(coords)) / 4, 4, 12))
        A = build_spatial(coords, mode="knn", k=k)
        return self._prepare(A)

    def loss(self, x, graph, epoch=0):  # decision 3: the objective
        z = self.encode(x, graph)
        recon = F.mse_loss(self.decoder(z), x)
```

```

z2 = self.encode(x + 0.1 * torch.randn_like(x), graph) # "consistency"
cons = F.mse_loss(z2, z.detach())
return recon + 0.5 * cons, {"recon": float(recon), "cons": float(cons)}, z

```

Thirteen lines, and it inherits the training loop, the refinement, the clustering and the ablation harness. That leverage is the entire argument for the base class in §4.2: the port is now *comparable* to the four reference methods by construction, because everything they share is shared code rather than four re-implementations that differ in ways nobody has audited.

Then, before believing anything:

```

results = ablate(Lambda: PaperMethod(n_genes, cfg), batch, A, evaluate, seeds=range(5))
print(report(results))

```

7.3 A release checklist

If the code is going out with a paper, the following are cheap and are what makes a reimplementaion match:

- **Pin the environment.** torch, torch-geometric, scanpy, numpy versions in a lockfile. scatter_reduce_semantics and KMeans defaults have both changed under people mid-project.
- **Ship the graph, not just the builder.** Save the adjacency alongside the results. It makes the graph auditable and removes the dependence on a neighbour-finding implementation.
- **Seeds and spread.** Five runs, mean and standard deviation, in the table.
- **Config files, not argparse defaults buried in main.** Serialise the config into the run directory (§2.3).
- **The permuted-graph ablation.** §6.3. If it is in the repository, reviewers can see it; if it is only in your notebook, it did not happen.
- **State what was tuned and against what.** Especially the number of clusters and the Leiden resolution.
- **A test that runs in under a minute.** `pytest -q tests/` on twenty fake nodes. It is the difference between a repository someone can build on and one they fork and abandon.

7.4 Ten things worth carrying away

1. Three lines — gather, transform, scatter — carry every architecture in this literature; only the middle one changes.
2. Normalisation belongs to the graph, not the layer; compute $\hat{A}$ once.
3. Pick one graph spelling per layer. Sparse matmul when the coefficient is fixed, edge list when the model computes it.
4. GATv2 needs two projections and the nonlinearity applied to their sum. Test the neighbour-ranking property; do not trust the code to be what it is named.
5. A likelihood that does not integrate to one is not a likelihood, and nothing in the training loop will tell you.
6. Assert `k1 > 1e-3` and `grad_norm > 0` at every logging step. Both failures are otherwise silent.
7. Self-referential objectives — KL, DEC, adversarial — all need warm-ups, for the same reason.

8. Neighbour sampling helps only when $n \gg Bs^L$; below that it costs a biased gradient and buys nothing.
9. A GNN is not bit-reproducible on GPU. Report five seeds and a spread, always.
10. The permuted-graph ablation is ten lines and is the experiment most likely to change your conclusion. Run it first.

Appendix

A. API reference

scmgnn.ops — `scatter_sum(src, index, n)`, `scatter_max`, `segment_softmax(e, index, n)`, `to_torch_sparse(A)`, `to_edge_index(A)`, `add_self_loops`, `dirichlet_energy(H, A_hat)`.

scmgnn.graphs — `build_knn(Z, k, mutual, weighted)`, `build_spatial(coords, mode, r, k, l)`, `build_spagcn_graph(coords, image_stat, l)`, `build_guidance(pairs, n_genes, n_peaks, signs)`, `build_psn(X, target_degree)`, `normalise(A, mode)`, `graph_report(A, labels)`, `permute_graph`, `random_graph`, `empty_graph`.

scmgnn.layers — `GCNLayer`, `GCNLayerEdge`, `GATLayer(v2=True)`, `GINLayer`, `SAGELayer`, `APPNPProp`, `grad_reverse`.

scmgnn.blocks — `MLPDecoder`, `GNNDecoder(kind='gcn' | 'gat' | 'appnp')`, `ZINBDecoder`, `InnerProductDecoder`, `DECHead`, `Discriminator`, `reparameterise`.

scmgnn.losses — `nb_nll`, `zinb_nll`, `gaussian_kl`, `graph_bce`, `signed_graph_bce`, `dgi_loss`, `info_nce`, `dec_kl`, `cox_ph`.

scmgnn.models — `CellGraphAE/CellGraphConfig`, `GuidanceGraphVAE/GuidanceConfig`, `SpatialGNN` with `SpaGCN`, `STAGATE`, `GraphST`, `SEDR` and `REGISTRY`.

scmgnn.train — `train(model, batch, graph, cfg)`, `TrainConfig`, `set_seed`, `smoothing_probe`.

scmgnn.ablate — `ablate(...)`, `verdict(results)`, `report(results)`.

The model contract, in one line: `loss(batch, graph, epoch) -> (scalar, terms_dict, embedding)`. Anything satisfying it works with `train` and `ablate`.

B. Equation index

#	What	Section
1	the four tensors	0.1
2	sparse matmul $\leftrightarrow$ gather/scatter	0.3
3	the MPNN template	1.1
4	GAT and GATv2 scores	1.4
5	GIN and SAGE	1.5
6	APPNP propagation and its fixed point	1.6
7	relational GCN with basis decomposition	1.9
8	guidance-graph decoder	3.1
9	guidance-graph objective	3.1
10	gradient reversal	3.4
11	SpaGCN's three-coordinate kernel	4.2
12	Deep Graph Infomax	4.2

#	What	Section
13	the ZINB log-likelihood, both branches	6.2

C. Numbers quoted in this document

Every figure below is produced by `verify/run_checks.py` on the machine that built this PDF; re-running it regenerates them.

Quantity	Value	Where
dense vs sparse propagation, $n = 8000$	$26\times$ slower	§1.7
dense vs sparse adjacency memory, $n = 16000$	$653\times$ larger	§1.7
GAT neighbour-ranking disagreements	0 / 8387	§1.4, §6.2
GATv2 neighbour-ranking disagreements	760 / 8387	§1.4, §6.2
ZINB total probability mass, correct signs	1.0000000000	§6.2
ZINB total probability mass, one sign flipped	1.1740	§6.2
2-hop reach, $n = 5000$, $k = 15$, 128 seeds	4981 nodes (100%)	§5.3
same, with fanouts (10, 5)	3220 nodes	§5.3
InfoNCE negatives sharing the query's label	34.6%	§1.5 note

D. Where the three volumes meet

Question	Volume
What did this paper claim, and who benchmarked it?	<i>Reading List</i> , §1–9 and §II
Why does this architecture work, and when does it fail?	<i>Techniques</i> , Parts A–E
What are the tensors, and what is the code?	this volume

The cross-reference markers are *[RL §x]* for the reading list and *[T §x]* for the techniques companion; both live in sibling folders of this one.

E. Reproducing everything

```
# numbers and checks – needs only numpy, scipy, scikit-learn, networkx
python3 verify/run_checks.py

# figures – additionally needs matplotlib
python3 figures-src/c04_benchmark.py      # regenerates from results.json
for f in figures-src/c*.py; do python3 "$f"; done

# the document
python3 build.py                        # pandoc + xelatex -> md and pdf

# the unit tests – needs torch
pytest -q tests/
```

verify/run_checks.py writes verify/results.json, which c04_benchmark.py reads, so the benchmark figure always shows the machine it was built on rather than numbers copied from somewhere else. The unit tests were compile-checked but not executed in the environment that produced this document, which had no PyTorch installed; run them once on your machine before trusting the model code.